

## ***Taken my prior essay “A Study of Ergonomic Peripherals”***

**This essay is out of date—it was written more than 4 years ago, before I began research in earnest.** It is retained for archival purposes, and may be insightful for gleaming how these efforts began.

[ IsopodOverseer, circa 2022 ]

Although there is much debate and experimentation with ways to improve keyboard efficiency, I have yet seen good dedicated, self-driven research on the matter of improving the mouse.

- Mice are used to interact with GUI.  
Keyboard interacts with data entry and terminal interfaces.
- Most modern navigation is GUI-driven. The mouse is a navigation device, but we navigate all the time during data-intensive tasks, especially those tasks which require multi-tasking and/or multi-monitor approaches.
- Mice are good especially for gaming and web browsing, but they need not be so limited in their positives.
- Increasing the speed we navigate Google helps us answer our questions faster.
- Increasing the speed at which we navigate menus, documents, and perform a routine series of clicks will be useful.
- The mice are often cheaper than keyboards, so buying different quality brands is more affordable than buying a quality keyboard, however, direct hardware customization (modding) may be more difficult.
- There are several mice on the market designed with multiple, programmable buttons. They are quite expensive, however.
- The mouse is the perfect vessel to attach some of the most commonly used keyboard shortcuts, especially those which are designed to be used alongside the mouse, necessitating the exchange of the hand from the mouse to the keyboard when alternating between navigation and entry, which can be hence avoided by judicious keybinding.

Let me ramble on a bit to introduce my system for categorizing the time savings each keybind is associated with. Furthermore, let me lay out my choices in procuring an ideal mouse for my experiments. You may wish to skip this section if it suits you.

Mice keybinds can reduce a number of inconveniences inherent to using this type of peripheral input device:

## 1. Pointer-Positioning Inconveniences

Pointing and clicking is a task which requires aim.

Aim is usually a very muscle-memorized automatic routine for those who are familiar with mice, but it still does require some small conscious effort.

For example, we aim when we:

- **PI#1** Move the text carat by clicking on a word in a text/code editor. If you want to select the back or front of a word, you need to aim properly, you need to put in a bare minimum level of effort.
  - *Counterpoint: You can always readjust with the arrow keys if you've missed.*
    - Counter-counterpoint: You need to transition to the keyboard, which you might do anyway, but you need to transition again from the arrow keys to a typing posture.
- **PI#2** Click and drag to a string of words in a text/code editor.
  - *Counterpoint: If you want to select the entire word you can double click. If you want the entire line you can triple click.*
    - Counter-counterpoint: But if you want a partial line or multiple lines, then you need to aim at where to start and stop.
- **PI#3** Click the minimize, full screen, and exit buttons on any program.
  - *Counterpoint: There are keyboard shortcuts for these, but they are not easier to hit than aiming outright and clicking. However, these do tend to always be in the top right corner, so you don't really need to go searching for them, not really too big an issue.*
    - Counter-counterpoint: When you resize your windows differently or have a custom grid arrangement of windows on the same screen, it does require effort to find and click these.
- **PI#4** Close and open new browsing tabs.
  - *Counterpoint: Keyboard shortcut is faster than aiming and clicking. We can just use that.*
    - Counter-counterpoint: But what if it could be faster? Ctrl + Shift + T is hard to reach.
- **PI#5** Reposition and navigate between existing browsing tabs. You need to aim and click every time, and the size of the tab decreases as you have more open, giving you a smaller target.
  - *Counterpoint: Keyboard shortcuts exist for those. Why have mouse ones?*
    - Keyboard shortcut is too cumbersome (chrome default is Ctrl + Tab, and Ctrl + Shift + PgDn/Up). Moves the hand out of typing position.

## 2. Scroll and Click Actuation

Actions which are ubiquitously used and very often used are clicking and scrolling. These actions should be as effortless as possible to perform to reduce strain and increase speed. Actions that consist of multiple repeated inputs of a similar nature can surely be automated. For example, repeated downscroll can be instead replaced by a keybind that takes you to the end of the page on a web browser, although this is not very optimal.

- **SCA#1** Reduce actuation distance of scrolling and mouse triggers. Reduce effort it takes to input these basic actions.
- **SCA#2** Automate repeated inputs of the same button or reassign something else to accomplish the intended task better.

### 3. Repositioning Inconveniences

Typically speaking, users generally have 3 “postures” of the two hands, or more if we count the “gaming posture” of placing left hand on WASD and other variants. We focus on the 3 following:

1. **Navigation Mode:** Right hand is on the mouse. Left hand is idle or in typing position.
2. **Typing Mode:** Right and left hands are in typing position.
3. **Text Navigation Mode:** Left hand is in typing position, right hand is to the lower right on the arrow keys to navigate between words. Or, the right hand can also be reaching right to access backspace and enter keys comfortably.

In my philosophy, time savings can be accomplished by reducing the time spent transitioning between these postures. 2 notable posture transitions are noted:

- Navigation <-> Typing. This occurs when scrolling a document, finding a section we wish to edit, and moving over to edit, then going back to scrolling.
- Typing <-> Text Navigation. This occurs when we make a mistake, need a minute carat adjustment, or are repositioning.

Issues arise when a sequence of posture transitions occurs non-optimally. For example, consider the following sequence of tasks:

4. Find an error in a document during scrolling
5. Correct the error, but in doing so, accidentally type something wrong
6. Upon examination, notice the wrong you have committed. The second to last word of the sentence you wrote is **compeleety** wrong. Navigate to the incorrect word.
7. Fix it, then resume scrolling the document

Consider the above sequence of tasks I've written as my document. In order to fix this mistake, I would transition like so:

1. Find an error in a document during scrolling (navigation – right hand on mouse)
2. Correct the error, but in doing so, accidentally type something wrong (typing – both hands on keyboard)
3. Upon examination, notice the wrong you have committed. The second to last word of the sentence you wrote is ~~compeleety~~ wrong. Navigate to the incorrect word. (text navigation – right hand on arrow keys, or backspace, or other small adjustment in carat that does not necessitate mouse aiming)
4. Fix it (typing – both hands on keyboard)
5. Resume scrolling the document (navigation – right hand on mouse)

This sequence is not optimal, but it is generally acceptable because there is a justification for every position. The distance the right hand must travel to interchange between typing posture and the arrow-keys/backspace is smaller than going for the mouse, hence the utility of arrow keys in general. However, the transition from mouse to typing is too large for simple tasks like adding indents and backspaces.

- **PI#1** Reduce the inconvenience of transitioning from mouse to typing where only a few select key-presses are needed (backspace and enter to rearrange structure, home and end to navigate faster or select segments, keyboard shortcuts).
- **PI#2** Implement fully functional text navigation and selection directly from mouse to lessen or replace the use the arrow keys and modifiers directly. Will assist in cases where transitions to the arrow key posture is slower than directly using mouse